COS 226 Assignment 2 - Experimental Results

Student numbers: uXXXXXXXX, uXXXXXXXX, uXXXXXXXX

Setup

Every run processes 120 000 jobs, divided equally between the producers (30 000 each with 4 producers, 20 000 each with 6, 60 000 each with 2). The bounded queue has a capacity of 100. Each configuration was run once as warm-up and then 5 times; the table shows the average. Every run checked that each job was processed exactly once.

Results

Queue	Producers	Consumers	Time (s)	Throughput (jobs/s)
Bounded	4	4	0.0329	3 644 158
Lock-free	4	4	0.0105	11 394 546
Bounded	6	2	0.0352	3 408 418
Lock-free	6	2	0.0139	8 634 996
Bounded	2	6	0.0347	3 459 544
Lock-free	2	6	0.0096	12 517 112

Observations

Balanced (4 producers, 4 consumers): the bounded queue sends every enq and deq through one lock, so all 8 threads compete for it. The lock-free queue only retries when two threads collide on head or tail with compareAndSet(), so it did much better.

Producer-heavy (6 producers, 2 consumers): consumers are the bottleneck, so the bounded queue is usually full and producers wait on notFull, which means threads are put to sleep and woken again. The lock-free queue is unbounded so producers never wait, but this also means the queue can grow without limit.

Consumer-heavy (2 producers, 6 consumers): producers are the bottleneck, so the queue is usually empty. In the bounded queue consumers sleep on notEmpty. In the lock-free queue deq() returns null on an empty queue, so consumers keep retrying (busy waiting), which wastes CPU but was still faster here.

Conclusion: the lock-free queue was faster in all three configurations, but the size of the difference changed with the workload. The bounded queue is slower but limits how far producers can get ahead and does not waste CPU when empty. The lock-free queue avoids lock contention but busy-waits when empty and has no limit on its size.